Árboles y métodos de ensamble

Tutorial 6 – De un árbol a un bosque

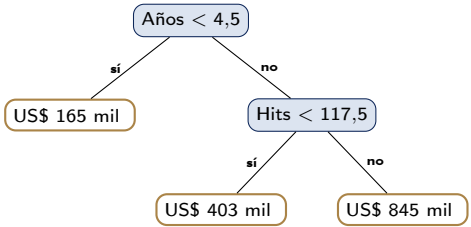
Luciana Azul Ramirez

PE1333 – Machine Learning para Economistas
Universidad de San Andrés

5 de noviembre de 2026

Qué es un árbol

- Un árbol predice con preguntas de sí o no, como en el *¿Quién es quién?*. Las preguntas que más separan van primero.
- Cada camino termina en una hoja, que predice el promedio de Y de los que llegaron ahí.
- En Hitters, con menos de 4,5 años en la liga los hits no cambian la predicción. Con más, sí. Esa interacción no la escribió nadie.



Salario de jugadores de béisbol (ISLP, cap. 8).

Test de lectura

T7.2. ¿Cómo le explicarías a alguien no técnico cómo funciona el ejemplo de Hitters y en qué sentido es no lineal?

Ejercicio

Seis jugadores, con sus años en la liga y su salario en cientos de miles de dólares.

Años	1	2	3	6	8	10
Salario	1	2	1	5	6	7

1. Si el árbol puede hacer un solo corte en los años, ¿dónde lo harían?
2. ¿Qué salario predice para un jugador con 5 años en la liga?
3. ¿Cuánto vale la suma de cuadrados si el árbol tiene una hoja por jugador?

Lo que hace el árbol

Prueba todas las variables y todos los cortes, y elige el que deja la menor suma de cuadrados dentro de las dos hojas.

Cuándo parar

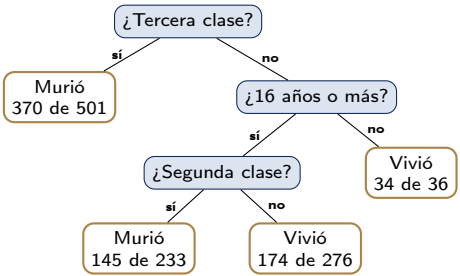
- La poda funciona como LASSO: se minimiza el ajuste más α por la cantidad de hojas. α hace el papel de λ.
- Con $\alpha = 0$ queda el árbol entero. Con α muy grande, una sola hoja con el promedio de todos.
- α se elige por cross validation.

Test de lectura

T7.5. En regresión, la complejidad la da la cantidad de regresores. ¿Y en CART? T7.6. ¿Por qué podar el árbol aumenta la impureza en cada paso de la poda?

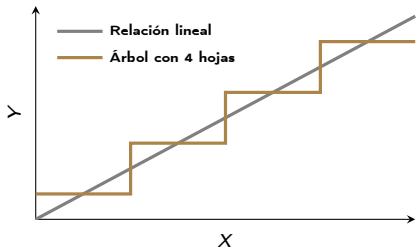
¿Los niños primero?

- Varian (2014) predice quién sobrevivió en el Titanic con dos variables: la clase y la edad.
- Una logit de supervivencia contra edad dice que la edad casi no importa ($p = 0{,}07$).
- El árbol dice que depende: los menores de 16 que no viajaban en tercera se salvaron casi todos.
- En clasificación, cada hoja predice la clase mayoritaria.



Lo bueno y lo malo de un árbol

- Encuentra no linealidades e interacciones solo, y se le explica a cualquiera.
- Si la relación es lineal anda mal.
- Es inestable. Si cambian unos pocos datos puede cambiar el primer corte, y con él todo el árbol.



Test de lectura

T7.9. ¿Qué tipo de problema resuelven bagging, forests y boosting?

¿Por qué métodos de ensamble?

- Vimos que CART captura no linealidades e interacciones *sin* imponer forma funcional, pero pagamos un costo: el árbol es **inestable**.
- "Inestable" quiere decir: si cambiamos un poquito la muestra de entrenamiento, el árbol estimado *cambia mucho*. En lenguaje del trade-off *sesgo–varianza*: **sesgo bajo, varianza alta**.
- Los *métodos de ensamble* parten de una idea simple de la estadística: **promediar reduce varianza**. Si tengo B predictores independientes con varianza σ^2, el promedio tiene varianza σ^2/B.
- Estrategia: en lugar de un solo árbol, entrenamos **muchos** sobre versiones perturbadas de la muestra y los promediamos.

Bootstrap: el ingrediente para fabricar muestras

- Dada una muestra de tamaño n, una **muestra bootstrap** es una muestra de tamaño n tomada *con reposición* de la original.
- Consecuencia: cada muestra bootstrap deja afuera $\approx 1/e \approx 37\,\%$ de las observaciones originales (las *out-of-bag*, ver más adelante).
- Repitiendo el procedimiento B veces, tenemos B muestras *distintas* (pero correlacionadas) que sirven para entrenar B modelos.

Bootstrap simula variabilidad muestral sin tener que recolectar más datos: es el motor de todos los ensambles basados en árboles.

Bagging: *Bootstrap Aggregating*

Entrenamos B árboles, cada uno sobre una muestra bootstrap distinta, y promediamos sus predicciones.

Algoritmo en tres pasos

1. Para $b = 1, \ldots, B$: tomar una muestra bootstrap Z^{*b} de la muestra original.
2. Sobre Z^{*b}, entrenar un árbol **sin podar** (alta varianza, bajo sesgo): $\hat{f}^{*b}$.
3. Combinar:

$$\hat{f}_{\text{bag}}(x) \;=\; \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x) \quad \text{(regresión)}$$

   o *voto mayoritario* entre las clases predichas (clasificación).

- Cuanto más *distintos* sean los árboles entre sí, más reduce la varianza el promedio.
- ¿Cuántos árboles? B no es un hiperparámetro a calibrar: cuanto más grande, mejor ($B \in [100, 500]$).

Test de lectura

T7.11. ¿Cómo se computa el error de predicción usando out of bag?

¿Por qué bagging no alcanza?

El problema: los árboles están correlacionados

Si hay un predictor **muy fuerte**, *todos* los árboles tienden a partir primero por esa variable. Árboles parecidos $\Rightarrow$ promediar gana *poco*.

- Con correlación promedio ρ entre árboles, la varianza del promedio es:

$$\rho\sigma^2 \; + \; \frac{(1-\rho)}{B}\sigma^2.$$

- Cuando $B \to \infty$ el segundo término se va a cero, pero el primero queda fijo en $\rho\sigma^2$. **Para bajar más la varianza hay que reducir** ρ. Esa es la motivación de *random forests*.

Test de lectura

T7.10. ¿Por qué si hay un predictor fuerte bagging no produce una auténtica reducción en la varianza del predictor?

Random Forest

En cada *split* de cada árbol no se considera todo el conjunto de predictores, solo un **subconjunto aleatorio de** $m < p$ variables.

¿Por qué funciona?

Forzamos a que los árboles "miren para otro lado" cuando construyen los splits. Las variables más fuertes ya no monopolizan el split raíz $\Rightarrow$ los árboles quedan **más distintos entre sí** (menos correlacionados) $\Rightarrow$ promediar reduce más la varianza.

- Convenciones (Breiman):
  - **Clasificación**: $m \approx \sqrt{p}$.
  - **Regresión**: $m \approx p/3$.
- Casos extremos: $m = p$ recupera *bagging* (no hay sorteo); $m = 1$ da splits casi aleatorios (máxima decorrelación, árboles muy débiles).
- En la práctica, m se elige por **cross-validation**.
- **Random Forest**: *flexibilidad*, *modelo de caja negra*, captura *no linealidades*.

Test de lectura

T7.12. ¿Cómo es que random forest resuelve el problema anterior?

Feature importance: interpretabilidad recuperada

Un árbol individual era *interpretable* (se lee como reglas *if-then*). El bosque, en cambio, son cientos de árboles promediados: no se "lee".

Variable importance scores

Para cada predictor X_j se calcula la **reducción promedio de impureza** (Gini en clasif., suma de cuadrados en regresión) que produce cada vez que aparece en un split, sumada sobre todos los árboles.

- Da un *ranking* de variables: cuáles son las que más usa el bosque.
- El score es **relativo**. Mide *poder predictivo*, no *importancia económica*.
- Es la herramienta clásica para *interpretar* un random forest.

Boosting

- **AdaBoost:** después de cada árbol los mal clasificados multiplican su peso por $(1 - \mathrm{err}_m)/\mathrm{err}_m$.
- Cada árbol nuevo presta más atención a los casos que los anteriores no lograron acertar.
- Al final vota cada árbol, con peso $\alpha_m = \ln[(1 - \mathrm{err}_m)/\mathrm{err}_m]$. El que se equivocó menos vota más, y si el error es 0,5 no vota.
- **Gradient boosting:** cada árbol predice los residuos del anterior.
- **XGBoost:** gradient boosting con regularización explícita, que castiga la cantidad de hojas y los pesos grandes de cada árbol. Además, es mucho más rápido.

En *bagging* los árboles se entrenan **independientemente y se promedian**. En *boosting* se entrenan **en secuencia, cada uno aprendiendo de los errores del anterior**.

Test de lectura

T7.14. Intuitivamente, ¿cómo hace boosting para ajustar los ponderadores en cada iteración?

CART vs. Bagging vs. Random Forest vs. Boosting

	CART	**Bagging**	**Random Forest**	**Boosting**
¿Cuántos árboles?	Uno	B árboles	B árboles	M árboles, en fila
¿Muestra de entrenamiento?	La original	B muestras bootstrap	B muestras bootstrap	La original, con pesos que cambian
¿Predictores en cada split?	Todos los p	Todos los p	$m < p$ sorteados	Todos los p
Sesgo	Bajo	Bajo	Bajo	Bajo
Varianza	Alta	Media	**Baja**	Baja, si M se elige bien
Interpretabilidad	Alta (reglas)	Baja (sólo *importance*)	Baja (sólo *importance*)	Baja (sólo *importance*)

- En **predicción**, RF y boosting suelen ganarle a bagging, que a su vez le gana a un único CART.
- En **interpretación**, el orden es al revés: CART se lee, los ensambles no.
- La pregunta práctica es: *¿necesito predecir o necesito explicar?* Las dos cosas son legítimas en economía aplicada.

Árboles en Python

En sklearn	Qué es	Por defecto
`DecisionTreeClassifier(ccp_alpha)`	α de la poda	0, sin podar
`RandomForestClassifier(max_features)`	m	$\sqrt{p}$
Con `max_features=None`	Bagging	
`oob_score=True`	Error out of bag	No lo calcula
`GradientBoostingClassifier(n_estimators, learning_rate)`	M y ν	100 y 0,1
`AdaBoostClassifier`	AdaBoost	50 árboles de un corte

- `cost_complexity_pruning_path()` da la grilla de α para el TP3.
- A los árboles no les importa la escala de las variables: no hace falta estandarizar.

Referencias

- Sosa Escudero, W. (2026). *CART's* y *Bagging, random forests y boosting*. PE1333, UdeSA. Test de lectura *CART, bagging, forests, boosting*.
- James, G., Witten, D., Hastie, T., Tibshirani, R. y Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*, cap. 8.
- Hastie, T., Tibshirani, R. y Friedman, J. (2009). *The Elements of Statistical Learning*, 2.ª ed., secciones 8.7, 9.2, 10.1 y 15.2.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Freund, Y. y Schapire, R. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- Varian, H. (2014). Big data: New tricks for econometrics. *Journal of Economic Perspectives*, 28(2), 3–28.